

LIGHTNINGBI - SESSION SUMMARY (2026-04-15)

DECISIONE STRATEGICA: MIGRAZIONE KOTLIN + GRADLE

Cambio di paradigma: Progetto migrato da Java/Maven a Kotlin/Gradle come opportunità di apprendimento.

PROGETTO COMPLETATO OGGI

1. MIGRAZIONE BACKEND

Azioni:

- Cancellato backend Java precedente
- Generato nuovo progetto con Spring Initializr:
 - Project: Gradle - Kotlin
 - Language: Kotlin
 - Spring Boot: 4.0.5
 - Java: 21
 - Dependencies: Web, Vaadin, Redis, Security, JDBC, Flyway, Validation
- Driver ClickHouse aggiunto: `com.clickhouse:clickhouse-jdbc:0.6.5:http`

Struttura finale:

```
lightningbi/
├── backend/
│   ├── lightning-engine/           # Progetto Kotlin/Gradle
│   │   ├── src/main/kotlin/
│   │   ├── src/main/resources/
│   │   │   ├── application.yml     # Config dev/prod
│   │   │   └── db/migration/       # V1-V4 SQL migrations
│   │   └── build.gradle.kts
│   ├── config/
│   ├── docs/
│   ├── scripts/
│   └── docker-compose.dev.yml
```

2. CONFIGURAZIONE PRESERVATA

application.yml (da sessione precedente):

- Profile dev: localhost ClickHouse/Redis
- Profile prod: variabili ambiente

- Flyway abilitato
- Security config (BCrypt + pepper, MFA, rate limiting)
- Superset URL config

SQL Migrations preserve (V1-V4):

- V1: system_users, system_roles, system_permissions, system_sessions
- V2: system_audit_log, system_auth_events + materialized views
- V3: ClickHouse row policies + 3 DB users (direction_user, management_user, operations_user)
- V4: Security views (view_fatti_direction, view_fatti_operations)

3. GIT WORKFLOW

Commits eseguiti:

1. "Migrate to Kotlin + Gradle with SQL migrations"
2. "Remove old Java source files"
3. "Update .gitignore and build.gradle.kts"

.gitignore aggiornato:

```
.idea/  
*.iml
```

4. SETUP INTELLIJ

Ufficio:

- Progetto aperto e linked come Gradle
- Kotlin plugin installato
- Syntax highlighting funzionante
- Gradle sync completato
- Tasto play verde visibile

Casa:

- Git pull eseguito (GitHub Desktop)
- Progetto aperto in IntelliJ
- Java 21 scaricato e configurato (aveva Java 25)
- Kotlin plugin installato
- Gradle sync completato (3-5 minuti prima volta)
- IntelliJ aggiornato a 2026.1

Issue risolti casa:

- "Cannot find Java installation" → scaricato Java 21 da IntelliJ
 - "Code insight unavailable" → cliccato "Link Gradle project"
 - Gradle sync lento prima volta (normale - scarica tutte dipendenze)
-

KOTLIN LEARNING: ENTITIES IMPLEMENTATE

Approccio blueprint

User fornisce **1 esempio**, Andrea implementa gli altri seguendo pattern.

4 Data Classes create

User.kt:

```
kotlin

data class User(
    val id: UUID,
    val username: String,
    val email: String,
    val passwordHash: String,
    val lastLogin: LocalDateTime? = null,
    val failedAttempts: Int = 0,
    val lockedUntil: LocalDateTime? = null,
    val mfaEnabled: Boolean = false,
    val mfaSecret: String? = null,
    val recoveryCodesHash: String? = null,
    val active: Boolean = true,
    val createdAt: LocalDateTime = LocalDateTime.now(),
    val updatedAt: LocalDateTime = LocalDateTime.now()
)
```

Role.kt:

```
kotlin

data class Role(
    val id: UUID,
    val name: String,
    val description: String,
    val createdAt: LocalDateTime = LocalDateTime.now()
)
```

Permission.kt:

```
kotlin
```

```
data class Permission(  
    val id: UUID,  
    val name: String,  
    val description: String,  
    val category: String,  
    val createdAt: LocalDateTime = LocalDateTime.now()  
)
```

Session.kt:

```
kotlin  
  
data class Session(  
    val sessionId: String,  
    val userId: UUID,  
    val roleId: UUID,  
    val ipAddress: String,  
    val userAgent: String,  
    val createdAt: LocalDateTime = LocalDateTime.now(),  
    val expiresAt: LocalDateTime,  
    val lastActivity: LocalDateTime = LocalDateTime.now(),  
    val revoked: Boolean = false  
)
```

Kotlin concepts learned

Visibilità:

- `public` è default (non serve scriverlo)
- `private`, `protected`, `internal` espliciti quando servono

Tipi ritorno:

- SEMPRE obbligatori (tranne `Unit` che può essere omesso)
- `Unit` = `void` di Java (ma è un vero tipo)

Data classes:

- Zero boilerplate (no getter/setter/constructor)
- `val` = immutabile, `var` = mutabile
- Parametri default supportati
- Nullable types con `?`

Annotazioni JPA:

- ❌ NON usate! ClickHouse non supporta JPA/Hibernate

- Entities sono POJO puliti
- Mapping manuale con RowMapper in Repository

Naming convention:

- SQL: `created_at` (snake_case)
 - Kotlin: `createdAt` (camelCase)
 - Repository farà conversione
-

REPOSITORY LAYER: COMPLETATO

4 Repository implementate

Pattern:

```
kotlin

interface UserRepository {
    fun findByUsername(username: String): User?
    fun findById(id: UUID): User?
    fun save(user: User): User
    fun update(user: User): User
    fun delete(id: UUID): Boolean
    fun findAll(): List<User>
}

@Repository
class UserRepositoryImpl(
    private val jdbcTemplate: JdbcTemplate
) : UserRepository {
    // JDBC puro con RowMapper
    // Mapping manuale SQL → Kotlin data class
    // ALTER TABLE per UPDATE/DELETE (sintassi ClickHouse)
}
```

Repository create:

- `UserRepository` / `UserRepositoryImpl`
- `RoleRepository` / `RoleRepositoryImpl`
- `PermissionRepository` / `PermissionRepositoryImpl` (con `findByCategory`)
- `SessionRepository` / `SessionRepositoryImpl` (con `revoke`, `deleteExpired`)

Note importanti:

- `ALTER TABLE ... UPDATE/DELETE` = sintassi ClickHouse (non SQL standard)

- `firstOrNull()` per query che ritornano 0 o 1 risultato
 - UUID → `toString()` per ClickHouse
 - Timestamp nullable → `?.toLocalDateTime()`
-

STRATEGIA DUE SOFTWARE (CONFERMATA)

Software 1: LightningBI Engine (OPENSOURCE)

Repository: github.com/AndreaZallocco84/lightningbi **Licenza:** GPL v3 **Scope:** Motore BI associativo generico

Contiene:

- Security layer completo (V1-V4)
- Semantic layer associativo query-based (logica Verde/Bianco/Grigio)
- Integration ClickHouse + Redis + Superset
- ETL framework generico
- Area vendite come DEMO/ESEMPIO

NON contiene:

- Business logic specifica
- Schemi dati aziendali
- Integrazioni ERP proprietarie

Software 2: Implementazione Privata

Scope: Analytics 6 aziende gruppo

Contiene:

- LightningBI Engine come dipendenza
 - 8 aree dati complete (vedi sotto)
 - ETL Gamma TeamSystem ERP
 - View specifiche gruppo
 - Dashboard custom
-

ARCHITETTURA DATI: 8 AREE ANALISI (RIVISTE)

Le 14 aree originali ridotte a 8 realmente mappabili:

1. Conto Economico
2. Stato Patrimoniale

3. Rendiconto Finanziario
4. Dati Commerciali Dettagliati
5. Acquisti e Fornitori
6. Magazzino e Logistica
7. Produzione
8. Risorse Umane

Tagliate (non mappabili):

- KPI Finanziari → è derivato dalle prime 3
- Budget e Forecast → layer di analisi sopra le prime 3, non area dati separata
- Mercato e Competitors → fonte dati non disponibile
- Rischi e Compliance → non mappabile
- ESG → non mappabile
- Digital e Innovation → non mappabile

In backlog (community contributions):

- ESG
- Digital e Innovation

Dimensioni condivise (conformate) tra aree:

- Tempo (data, mese, trimestre, anno)
- Azienda/Business Unit (6 aziende gruppo)
- Articoli (condivisa tra vendite, acquisti, produzione)
- Clienti/Fornitori (condivisa dove applicabile)

Dimensioni locali per area:

- Macchine, centri di costo → Produzione
- Piano dei conti, voci di bilancio → Finance/CDA

MIGRATIONS PLAN

V1-V4: Security/Auth ☒ COMPLETATO

Prossime migrations (da fare):

- V5-V6: Vendite (area pilota per testare motore)
 - V7+: Altre aree dopo validazione motore
-

SEMANTIC LAYER ASSOCIATIVO QUERY-BASED

⚠ Terminologia corretta: **"semantic layer associativo query-based"**, NON "motore associativo". Cambia le aspettative, non l'architettura.

Concetto Fondamentale

Tre stati per ogni valore di ogni dimensione:

- **VERDE** = selezionato attivamente dall'utente
- **BIANCO** = alternativo (nel campo con selezione attiva) o associato (negli altri campi)
- **GRIGIO** = escluso, nessun record compatibile

Regola Qlik Corretta per gli Stati

Nel campo con selezione attiva:

- Verde = valori selezionati
- Bianco/grigio chiaro = valori alternativi (non selezionati ma presenti nel campo)

Negli altri campi:

- Verde = possibile (associato alla selezione corrente)
- Grigio = escluso (nessun record lo collega alla selezione)

Algoritmo "Omit Self" (cuore del sistema)

Per calcolare lo stato alternativo serve una query per campo che **esclude la selezione di quel campo stesso**. Non bastano 3 query generiche — serve una query per dimensione che omette il proprio filtro:

```
Per ogni dimensione D:  
query_stato_D = SELECT DISTINCT D FROM fatti  
                WHERE [tutti i filtri attivi ECCETTO filtro su D]
```

Questo è il meccanismo che permette di mostrare i valori alternativi nel campo selezionato.

Invalidazione Cache per Partizione

- ❌ Flush totale cache → troppo aggressivo
- ✅ Invalidazione per **partizione/data toccata dall'ETL**
- Redis keys strutturate per includere la partizione temporale

Roaring Bitmap (ottimizzazione avanzata)

ClickHouse ha `groupBitmap` / `bitmapAnd` nativi. Per dimensioni a bassa/media cardinalità: precalcolare bitmap per ogni valore → stati calcolabili con operazioni bitmap. È la risposta

reale al problema "Qlik senza RAM".

Tabelle Simboli

Dizionari per campo (valore → id intero) in ClickHouse = base per bitmap e confronto veloce. Avvicina il modello Qlik senza richiedere tutto in RAM.

Latenza Target

- < **200ms** per click con K listbox aperte
- Senza SLA definito non si può dimensionare correttamente

Debounce/Cancellazione Richieste

Click rapidi in sequenza generano query zombie. Il backend deve **cancellare richieste obsolete** prima di eseguire la nuova.

Superset e UI Associativa

- ⚠ Superset **NON renderizza** Verde/Bianco/Grigio
- La UI associativa deve essere **custom** (Vaadin o web component)
- Superset resta per **dashboard statiche**

Multi-Fact (decisione strutturale)

Con più tabelle fatti (vendite, acquisti, produzione...) serve definire ora come le associazioni attraversano le fact table:

- **Link table** stile Qlik, oppure
- **Concatenazione** delle fact table

⚠ Da decidere prima di implementare — dopo è refactoring doloroso.

Test Coerenza Associativa

Suite automatica che verifica stati Verde/Grigio contro risultati attesi su dataset noto. È la parte che si rompe silenziosamente senza test.

Bookmark / Selezioni Salvate

Feature attesa dagli utenti Qlik. Da includere nel backlog UI.

Monitoraggio Query per Stato

Loggare quale query stato-dimensione è lenta, non solo ">1s generico".

ARCHITETTURA DATI: AUTH SU POSTGRESQL

⚠ Correzione architetturale importante

- **Utenti, ruoli, sessioni** → PostgreSQL (non ClickHouse)
- ClickHouse è OLAP: non adatto a update frequenti né a credenziali
- Le migrations V1-V4 vanno su PostgreSQL

Con Keycloak già nello stack TrooperERP:

- Valutare uso Keycloak anche per LightningBI
- Evita reinventare auth (JWT refresh, revoca, logout, scadenza)
- Mancanza gestione refresh/revoca token è un gap attuale

TIERED STORAGE (FEATURE CORE)

Problema

IoT/sensori generano milioni di eventi/giorno. Memorizzare tutto per sempre fa esplodere il database.

Soluzione: 3 Livelli Automatici

Tier	Durata	Granularità	Storage
HOT	7 giorni	Massima (raw)	SSD veloce
WARM	3 anni	Aggregati orari/giornalieri	HDD standard
COLD	Storico	Export Parquet	S3/MinIO economico

Implementazione

- ClickHouse TTL automatico nelle migrations
- Auto-cancellazione/aggregazione ogni notte
- Zero manutenzione manuale

Nota su fatti_giacenze

- ❌ Snapshot giornaliero articoli × depositi × 365 → esplode
- ✅ Snapshot mensile + delta, oppure `AggregatingMergeTree`

IDEMPOTENZA ETL IN CLICKHOUSE

|  Correzione implementativa

- `ALTER TABLE DELETE` = mutazione **asincrona e pesante** in ClickHouse
 - ❌ `DELETE + INSERT` non è idempotenza affidabile
 - ✅ Alternativa 1: `ReplacingMergeTree` con colonna versione
 - ✅ Alternativa 2: partizioni giornaliere con `DROP PARTITION` + reinsert
-

DIMENSIONAMENTO INFRASTRUTTURA

- ⚠️ 3GB RAM ClickHouse è sottodimensionato appena si aggiunge giacenze/produzione
 - ✅ Partire da **8-16GB RAM** per ambiente realistico
-

ARCHITETTURA ENTERPRISE: KAFKA + DRUID

Stack Architetturale Finale

TIER 1 - INGESTION: Kafka Cluster, Kafka Connect, Schema Registry **TIER 2 - PROCESSING:** Kafka Streams, ksqlDB (optional) **TIER 3 - STORAGE:** ClickHouse (default), Druid (optional time-series), ClickHouse Kafka Engine **TIER 4 - CACHE:** Redis Cluster **TIER 5 - ANALYTICS:** LightningBI Engine, Query builder dinamico, Verde/Bianco/Grigio logic **TIER 6 - VISUALIZATION:** Superset (dashboard statiche), UI custom Vaadin (associativa)

Deployment Scale

Scale	Throughput	Costo cloud
Small	< 100K events/sec	~\$200/mese
Medium	< 1M events/sec	~\$1.500/mese
Large	> 1M events/sec	~\$5.000+/mese

PROSSIMI STEP

Scaletta aggiornata:

1. ~~Model~~ ✅
2. ~~Repository~~ ✅
3. **Service Layer** ← prossimo
4. **Config Layer** (SecurityConfig, BCrypt, Spring Security / Keycloak)

5. **ETL** (riempie fatti + dimensioni ClickHouse)
6. **Semantic Layer Associativo** (incluso Redis)
7. **UI Admin** (Vaadin - gestione utenti, trigger ETL)
8. **UI Dashboard Associativa** (Vaadin custom - Verde/Bianco/Grigio)
9. **Superset** (dashboard statiche)

Da decidere prima del Semantic Layer:

- Strategia Multi-Fact (link table vs concatenazione)
 - Migrazione auth su PostgreSQL
 - Integrazione Keycloak vs auth custom
 - Latenza target definita (es. < 200ms)
-

CONCETTI ARCHITETTURALI CHIARITI

ClickHouse vs Qlik

ClickHouse = Storage fisico (fondamenta):

- Star schema (fatti + dimensioni)
- Aggregazioni colonnari veloci
- Join ottimizzati

Logica Qlik = Semantic layer (casa):

- Verde/Bianco/Grigio dinamico con "omit self"
- Calcolo stati dimensioni via query
- WHERE clause dinamica

Senza star schema ClickHouse, logica associativa non funziona.

Docker Setup

Casa e Ufficio:

- Stesso docker-compose.dev.yml
 - ClickHouse + Redis + Superset locali
 - Sviluppo locale, NO connessione server produzione (per ora)
 - Server aziendale dopo Fase 1 completa
-

Ufficio: Windows 11, IntelliJ 2026.1, Java 21 (Eclipse Temurin), Git CLI, Docker Desktop

Casa: Windows 11, IntelliJ 2026.1, Java 21 + Java 25, GitHub Desktop (NO Git CLI), Docker Desktop

KNOWLEDGE BASE UTILE

- **White Paper Qlik:** <https://www.e-mergo.nl/wp-content/uploads/2017/11/WP-Qlik-Associative-Model-EN-Emergo.pdf>
 - **Spring Initializr:** <https://start.spring.io/>
 - **Gradle Kotlin DSL:** Plugin kotlin("jvm") 2.2.21, Spring Boot 4.0.5, Java toolchain 21
-

SESSIONI PRECEDENTI REFERENZIATE

2026-04-09: Security design completo (V1-V4 migrations, RBAC architecture) **2026-04-02:** ClickHouse setup, 14 aree analisi, strategia generale

Transcript disponibili in: /mnt/transcripts/2026-04-09-15-11-27-lightningbi-security-design.txt /mnt/transcripts/2026-04-15-18-44-41-kotlin-migration-gradle-setup.txt